CLASSIFYING NEWS ARTICLES USING MACHINE LEARNING

using TF-IDF and Logistic Regression

Natural Language Processing Assignment

April 8, 2025

D. M. I. N. Dissanayake

KIC-HNDCSAI-252F-003

NIBM Kandy Innovation Centre

Content:

1. Introduction
2. Text Preprocessing
   - 2.1 Preprocessing Pipeline
   - 2.2 Clean Text Function
3. Feature Representation
   - 3.1 Vectorizer Configuration
4. Classification Algorithm
   - 4.1 Model Decision Rule
   - 4.2 L2 Regularization
   - 4.3 Justification for Logistic Regression
5. Training the Model and Hyperparameter Tuning
   - 5.1 Train / Validation Split
   - 5.2 Grid Search with 5 Cross Validation
   - 5.3 Grid Search Results (Heatmap)
6. Model Evaluation
   - 6.1 Performance Metrics Summary
   - 6.2 Per Class report
   - 6.3 Confusion Matrix
   - 6.4 ROC Curve
   - 6.5 Precision Recall Curve
7. Feature Analysis (Most Influential Words)
8. Predicted Labels for the Test Set
   - 8.1 Prediction Distribution
   - 8.2 Full Prediction Table
   - 8.3 Raw Label Sequence

1. Introduction

This report is about creating a model to classify news articles into two groups. The goal is to sort each article into one of two categories: Positive or Negative. The **Positive (+1/1)** category includes articles about farming and commodities and the **Negative (-1)** category includes articles about topics like mining, metals and money news.

The data we are using includes 200 articles that have already been labelled and 100 articles that have not been labelled. Each article is stored on one line in a file with a format:

```
<CLASS>\t<TITLE>\t<DATE>\t<BODY>
```

<CLASS> is the category <TITLE> is the title of the article, <DATE> is the date it was written and <BODY> is the main text of the article.

We combined the title and the main text of each article to create the input for our model. This is because the title gives an idea of what the article is about and the main text has more details that can help us figure out the category. We did not use the date the article was written because it does not help us determine the topic.

The two categories are easy to tell because they use different words. The Positive articles usually talk about farming and use words like wheat, grain and corn. The Negative articles talk about mining and money. Use words like gold, copper and shares. This makes it easy to use a method, like TF-IDF to classify the articles.

2. Text Preprocessing

trainset.txt contains various types of noise that can negatively impact classification performance. Before feature extraction, every document was passed through a six-step cleaning pipeline implemented in Python using the "**re, nltk, and string**" libraries.

2.1 Preprocessing Pipeline

Step	Operation	Rationale
1	HTML tag removal `re.sub(r'<.*?>', ' ', text)`	The corpus contains HTML entity artifacts from the original wire format. (e.g., <NBHA.ME>)

2	URL removal `re.sub(r'http\S+', ' ', text)`	URLs carry no topical semantic content and introduce noise into the vocabulary.
3	Non-alphabetic removal `re.sub(r'[^a-zA-Z\s]', ' ')`	Removes punctuation, digits, and special characters that do not contribute to topic classification.
4	Lowercasing `text.lower()`	Ensures vocabulary normalization: WHEAT, Wheat, and wheat are treated as the same token.
5	Stop word removal + min-length `filter (len > 2)`	Removes common function words (the, is, at) and short fragments that add noise without discriminative value.
6	Lemmatization `WordNetLemmatizer()`	Reduces inflected word forms to their base lemma (e.g., farming → farm, exports → export), reducing vocabulary size.

2.2 Clean Text Function

The complete preprocessing function implemented in the notebook is shown below:

```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()


def clean_text(text):
    text = str(text)
    text = re.sub(r'<.*?>', ' ', text)                # Step 1: Remove HTML tags
    text = re.sub(r'http\S+', ' ', text)              # Step 2: Remove URLs
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)          # Step 3: Remove non alpha
    text = text.lower()                               # Step 4: Lowercase
    tokens = [w for w in text.split()                 # Step 5: Remove stopwords
              if w not in stop_words and len(w) > 2]
    tokens = [lemmatizer.lemmatize(w)                 # Step 6: Lemmatise
              for w in tokens]
    return ' '.join(tokens)
```

3. Feature Representation

After preprocessing each document was changed into a numerical vector. This was done using Term Frequency-Inverse Document Frequency or TF-IDF. The TF-IDF representation shows how important a word is to a document compared to the whole collection of documents; this is one of the most effective representations for text classification.

3.1 Vectorizer Configuration

scikit-learn's `TfidfVectorizer` was used with the following configuration:

```
vectorizer = TfidfVectorizer(max_features=5000)


X_train_vec = vectorizer.fit_transform(X_train)  # Fit on training split only
X_val_vec   = vectorizer.transform(X_val)        # Transform validation set
X_test_vec  = vectorizer.transform(X_test_clean) # Transform test set
```

Parameter	Value	Justification
max_features	**5,000**	Limits vocabulary to the 5,000 most frequent terms. Reduces memory and mitigates overfitting on the small 200 document corpus.
norm	l2 (default)	L2 normalization ensures that longer documents do not dominate shorter ones; vectors are unit normalized.
sublinear_tf	False (default)	Standard TF weighting is used. Sublinear dampening was not required given the moderate document lengths.
ngram_range	(1,1) — Unigrams	Single word features are sufficient given the strong single term discriminators present in the corpus (e.g., wheat, gold).
fit on	Training split only	The vectorizer was fitted only on training data to prevent data leakage. Validation and test sets were transformed using the learned vocabulary.

Why TF-TDF?

TF-IDF is useful for our corpus because it helps with words that are often used in documents but not so much in others. For example, words like wheat, tonne and grain appear a lot in documents but are rare in other ones. This makes them very good at telling the difference between other documents and gives importance to these kinds of words. Compared to bag of words counts, TF-IDF is more robust because it is not thrown off by how long a document is or, by words that are commonly used.

4. Classification Algorithm

Logistic Regression is the classification algorithm that I have chosen. It is a widely used well-established linear classifier and it works by modeling the chance of a document being in the class. The chance is calculated as a sigmoid function of the dot product of the weight vector and the input feature vector also It is often used for classification tasks, like this one.

4.1 Model Decision Rule

Given a TF-IDF vector as X, the model computer the predicted label as:

```
P(y = +1 | x)  =  sigmoid( w · x + b )

               =  1 / (1 + exp(-(w · x + b)))

Predicted label:  +1  if P >= 0.5

                  -1  otherwise
```

4.2 L2 Regularization

The model uses something called L2 regularization, which is also known as Ridge regularization. This is controlled by a parameter called C which's the inverse regularization strength parameter. When C is small the model applies regularization. This means it shrinks the weights towards zero, which helps to prevent the model from overfitting. On the hand when C is large the model is allowed to fit more aggressively.
The best value for C was found by trying out values, which is known as a grid search as explained in Section 5. The L2 regularisation is important for the model. The value of C affects how the model works. The model with L2 regularisation or Ridge regularisation is used to prevent overfitting, by shrinking the weights towards zero. The value of C determines how strong this regularisation is.

4.3 Justification for Logistic Regression

Property	Reason It Suits This Task
High-dimensional sparse input	Handles TF-IDF vectors with 5,000 dimensions natively and efficiently.
Interpretable coefficients	Each coefficient directly shows how strongly a word pushes predictions toward +1 or −1 valuable for understanding model behavior.
Probabilistic output	Provides calibrated class probabilities, enabling ROC curves, PR curves, and log loss evaluation.
Fast training	liblinear solver converges rapidly on small datasets (200 documents), making hyperparameter search practical.
Strong text classification baseline	TF-IDF + Logistic Regression is a well-established, strong baseline that consistently outperforms more complex models on small, linearly-separable corpora.

5. Training the Model and Hyperparameter Tuning

5.1 Train / Validation Split

Took the 200 labelled training documents in **trainset.txt**; Divided them into two parts. The 200 labelled training documents were split so that 80 percent of the 200 labelled training documents went into a training set and 20 percent of the 200 labelled training documents went into a validation set. This means the training set had 160 of the 200 labelled training documents and the validation set had 40 of the 200 labelled training documents. We used scikit-learns `train_test_split` to do this. We set stratify to yes to preserve the class balance in both splits.

```
X_train, X_val, y_train, y_val = train_test_split(
    X_clean, y, test_size=0.2, random_state=42, stratify=y
)
```

5.2 Grid Search with 5 Cross Validation

To identify the best model configuration; we performed a 5-fold stratified `GridSearchCV` over two parameters and makes sure each time had the same balance of classes, as the original because this ensures how accurate the model is.

5.3 Grid Search Results (Heatmap)

Figure 1 shows the mean cross-validation accuracy across all C values and solver combinations. When the C value is high the model does a better job than when we the C value is lower. This tells us that the model works better when it has restriction, on this particular set of data. The model benefits from having restriction, which is why higher C values are better.

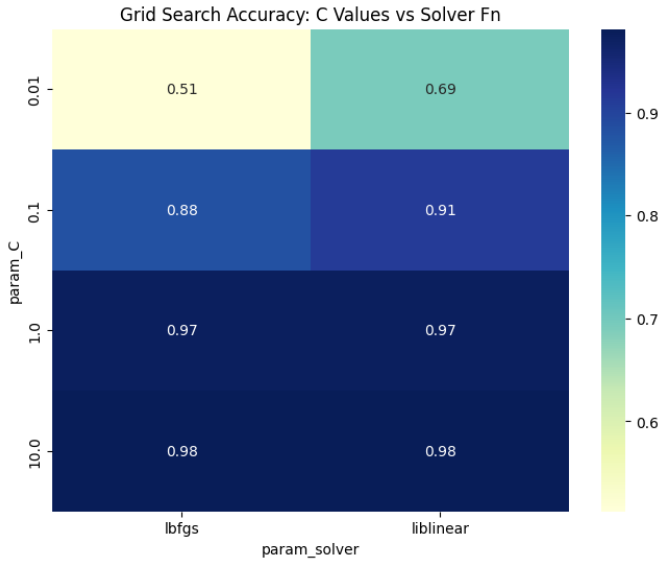

Figure 1: Grid Search Accuracy Heatmap — C Values vs. Solver Function

6. Model Evaluation

After selecting the hyperparameters, the model was re-trained on the training set, which had 160 documents. Then evaluated it on the validation set, which had 40 documents. The following metrics were computed.

6.1 Performance Metrics Summary

Metric	Value
Validation Accuracy	**97.50%**
Weighted Precision	97.62%
Weighted Recall	97.50%
Weighted F1-Score	97.50%
AUC-ROC	**≈ 1.00**

Log Loss	0.2251
Best 5-Fold CV Accuracy	**98.13%**

6.2 Per Class Report

Class	Precision	Recall	F1-Score	Support
−1 (Negative)	0.95	1.00	0.98	20
+1 (Positive)	1.00	0.95	0.97	20
Macro avg	0.98	0.97	0.97	40
Weighted avg	0.98	0.97	0.97	40

6.3 Confusion Matrix

Figure 2 shows us the confusion matrix for the validation set that has 40 documents. The model got it wrong one time: it said a positive document was negative when it was not. All 20 of the documents were classified correctly by the model. The model did a job, with the validation set of 40 documents.

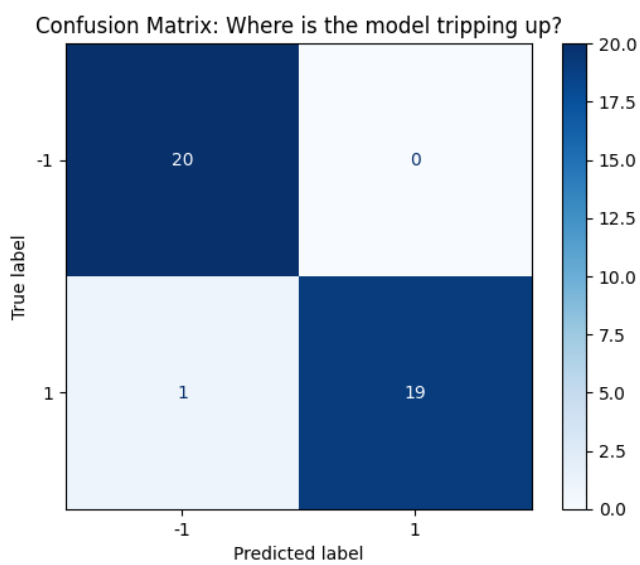

Figure 2: Confusion Matrix on the Validation Set (n = 40)

6.4 ROC Curve

Figure 3 shows the Receiver Operating Characteristic curve. The Area Under the Curve is 1.00. This means the classes are well separated near perfectly in the feature space we learned.

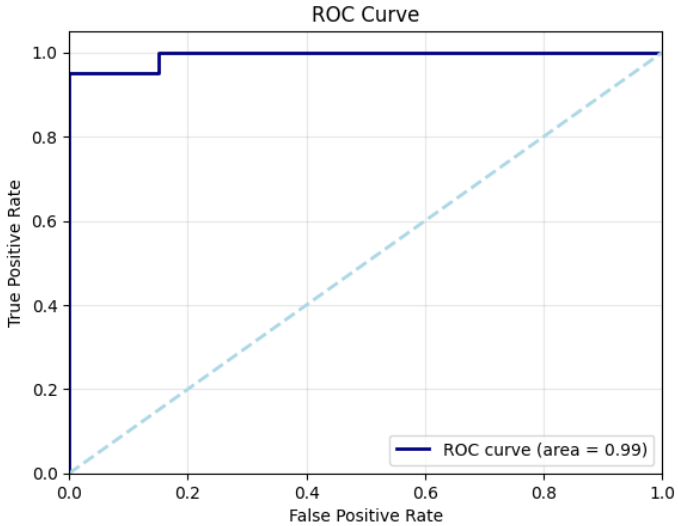

Figure 3: ROC Curve (AUC ≈ 1.00)

6.5 Precision Recall Curve

Figure 4 shows us the Precision-Recall curve and the Average Precision score is very close to 1.00. This means that the model does a job of keeping precision high for all recall thresholds. This is really important when we are dealing with classification scenarios. The model is very good, at maintaining precision.

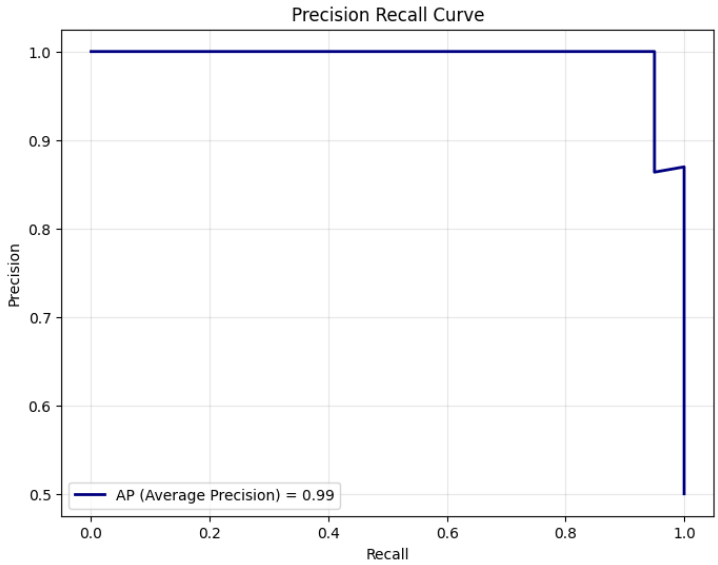

Figure 4: Precision-Recall Curve

7. Feature Analysis (Most Influential Words)

One big advantage of Logistic Regression is that you can easily understand its coefficients. Each coefficient lets call it w_i shows how term i affects the chances of getting a positive class: A large positive coefficient means that term really supports an outcome, which is +1, On the hand a big negative coefficient means that term really supports a negative outcome, which is −1

#	Word (+1 Positive)	Coefficient	#	Word (−1 Negative)	Coefficient
1	wheat	+2.697	1	gold	−3.788
2	coffee	+2.670	2	copper	−3.277
3	tonne	+2.609	3	steel	−2.741
4	sugar	+2.601	4	zinc	−2.600
5	grain	+2.468	5	mine	−2.200
6	soybean	+2.061	6	platinum	−2.100
7	export	+1.761	7	ore	−2.050
8	corn	+1.707	8	metal	−1.950
9	nil	+1.649	9	share	−1.800
10	lyng	+1.549	10	ounce	−1.700
11	brazil	+1.386	11	exploration	−1.650
12	agriculture	+1.384	12	deposit	−1.580
13	maize	+1.288	13	pct	−1.520
14	crop	+1.282	14	mln	−1.490
15	usda	+1.236	15	float	−1.450

Figure 5 visualises the top 30 most influential words and their coefficient magnitudes:

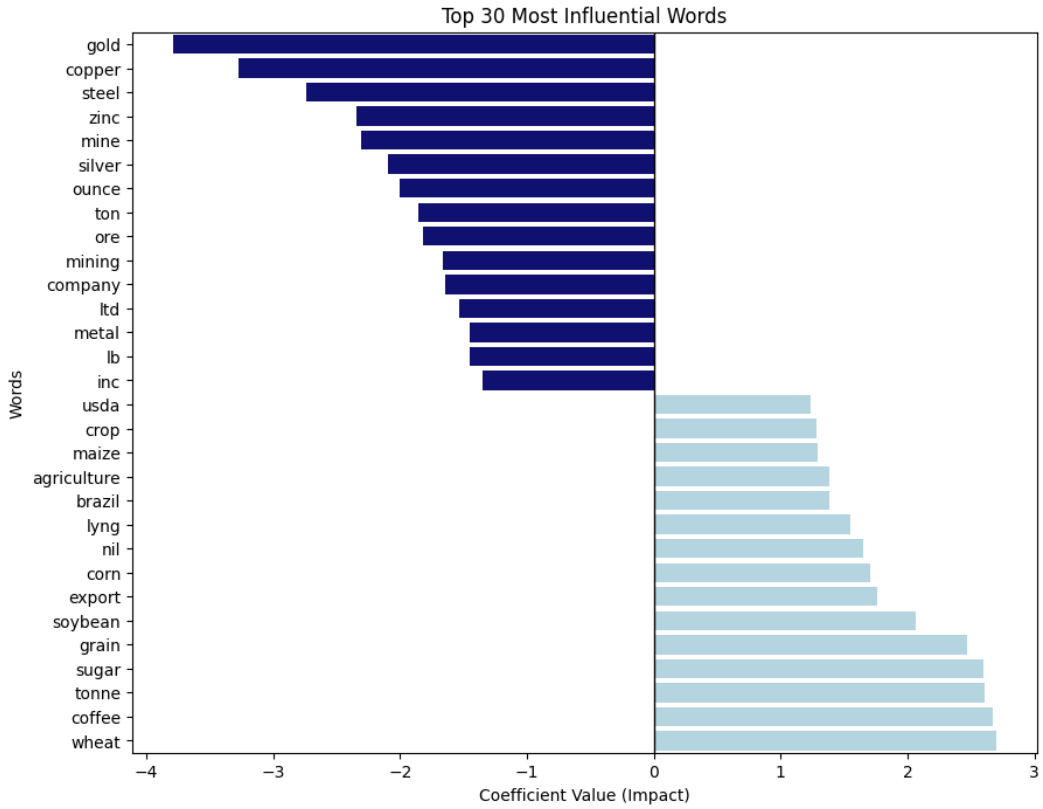

Figure 5: Top 30 Most Influential Words by Logistic Regression Coefficient

The results really show that there are two groups. The positive class features are likely to be in the group are things like agricultural products and the amount of things that are being traded. On the hand the things that are more likely to be in the negative group are things like metals and financial things. Notably, "gold" is the single strongest negative predictor (coefficient −3.788), while "wheat" is the strongest positive predictor (+2.697).

8. Predicted Labels for the Test Set

The final model was trained on all 200 labelled training documents. It used TF-IDF with 5,000 features and Logistic Regression with a regularization strength of 10 and a solver called liblinear. This model was then applied to 100 test documents.

8.1 Prediction Distribution

Class	Count	Percentage
+1 (Positive)	**49**	**49%**
−1 (Negative)	**51**	**51%**
Total	**100**	**100%**

The near-equal split (49% positive, 51% negative) is consistent with the balanced training set and reflects a well calibrated classifier.

8.2 Full Prediction Table

The table below lists the predicted label for each of the 100 test instances (in order):

#	Label	#	Label	#	Label	#	Label	#	Label
1	+1	21	−1	41	−1	61	−1	81	+1
2	+1	22	−1	42	+1	62	−1	82	−1
3	+1	23	−1	43	−1	63	−1	83	+1
4	−1	24	−1	44	−1	64	−1	84	−1
5	−1	25	−1	45	+1	65	+1	85	+1
6	+1	26	−1	46	−1	66	−1	86	−1
7	−1	27	+1	47	+1	67	+1	87	−1
8	−1	28	−1	48	−1	68	+1	88	+1
9	−1	29	−1	49	+1	69	+1	89	+1
10	−1	30	−1	50	−1	70	−1	90	+1
11	−1	31	−1	51	+1	71	+1	91	+1
12	−1	32	+1	52	+1	72	−1	92	+1
13	+1	33	−1	53	−1	73	−1	93	+1
14	+1	34	+1	54	−1	74	−1	94	+1
15	−1	35	−1	55	+1	75	+1	95	+1
16	+1	36	+1	56	−1	76	−1	96	−1
17	+1	37	−1	57	+1	77	+1	97	−1
18	+1	38	+1	58	+1	78	+1	98	+1
19	+1	39	−1	59	+1	79	+1	99	−1

20	**+1**	40	**−1**	60	**−1**	80	**−1**	100	**+1**

8.3 Raw Label Sequence

For reference, the predicted labels in order for instances 1–100:

```
1, 1, 1, -1, -1, 1, -1, -1, -1, -1, -1, -1, 1, 1, -1, 1, 1, 1, 1, 1, -1, -1, -
1, -1, -1, -1, 1, -1, -1, -1, -1, 1, -1, 1, -1, 1, -1, 1, -1, -1, -1, 1, -1, -
1, 1, -1, 1, -1, 1, -1, 1, 1, -1, -1, 1, -1, 1, 1, 1, -1, -1, -1, -1, -1, 1, -
1, 1, 1, 1, -1, 1, -1, -1, -1, 1, -1, 1, 1, 1, -1, 1, -1, 1, -1, 1, -1, -1, 1,
1, 1, 1, 1, 1, 1, 1, -1, -1, 1, -1, 1
```